

RELATÓRIO DO PROJETO MYFOOD – MILESTONES 01 E 02

Disciplina: Programação 2

Projeto: MyFood – Milestones 01 e 02

Linguagem: Java

Aluno: Artur Ferreira Marques da Silva

Professor: Mario Hozano Lucas de Souza

Período: 2026.1

1. Introdução

Este relatório apresenta a implementação dos Milestones 01 e 02 do projeto MyFood, desenvolvido na disciplina de Programação 2, utilizando a linguagem Java. O objetivo do projeto foi construir, de forma incremental, um sistema capaz de gerenciar usuários, empresas, produtos, pedidos, entregadores e entregas, respeitando as histórias de usuário definidas para os dois primeiros marcos da atividade.

No Milestone 01, o foco esteve na construção da base do sistema, incluindo cadastro de usuários, autenticação, cadastro de empresas, criação e manipulação de produtos, criação de pedidos e persistência em arquivo. No Milestone 02, o sistema foi expandido com novos tipos de empresa, como mercados e farmácias, além da introdução de entregadores, vínculo entre entregadores e empresas, liberação de pedidos e gerenciamento de entregas.

O sistema foi implementado de forma compatível com os testes de aceitação disponibilizados para o projeto, contemplando as funcionalidades cobradas até a US 8_2. Além disso, durante o desenvolvimento, buscou-se aplicar os principais conceitos estudados em sala de aula, especialmente os fundamentos de programação orientada a objetos, como encapsulamento, herança, abstração, polimorfismo, composição, uso de listas, tratamento de exceções e organização em pacotes.

Este documento descreve a arquitetura geral do sistema, seus principais componentes, a interação entre eles, os conceitos de orientação a objetos aplicados e o padrão de projeto utilizado.

2. Descrição geral do design arquitetural do sistema

O projeto foi organizado de forma modular, separando as responsabilidades em diferentes pacotes e classes. A arquitetura escolhida teve como objetivo principal facilitar a compreensão do sistema, evitar concentração excessiva de responsabilidades em uma única classe e tornar o código mais legível, modular e manutenível.

A estrutura principal do sistema foi dividida em:

- **models**: contém as entidades centrais do domínio do problema;
- **services**: contém classes responsáveis pela lógica de negócio e pela persistência;
- **exceptions**: contém a exceção personalizada utilizada no projeto;
- **Facade**: classe de acesso principal utilizada pelos testes de aceitação;
- **Main**: classe responsável por executar os testes via EasyAccept.

Essa organização permite uma separação clara entre:

- os dados do sistema;
- as regras de negócio;
- o tratamento de erros;
- a persistência em arquivo;
- e o ponto de entrada externo do sistema.

Em vez de concentrar toda a lógica em uma única classe, o sistema foi dividido em serviços especializados, cada um responsável por uma parte do domínio. Essa decisão torna o design mais coerente com os princípios de orientação a objetos estudados em sala, especialmente a ideia de delegar responsabilidades às classes adequadas.

Com a expansão do sistema no Milestone 02, essa organização se mostrou ainda mais importante, pois permitiu adicionar novos tipos de empresa e novos fluxos de entrega sem comprometer a base construída anteriormente.

3. Principais componentes e suas interações

3.1 Classe Usuario

A classe Usuario representa o conceito geral de usuário do sistema. Ela foi implementada como classe abstrata, pois não faz sentido instanciar um usuário genérico. O sistema trabalha, de forma concreta, com diferentes tipos de usuário, cada um com suas características e responsabilidades próprias.

Essa classe centraliza os atributos comuns entre os usuários:

- id
- nome
- email
- senha
- endereço

Além disso, define métodos abstratos relacionados ao tipo concreto do usuário, permitindo distinguir seu papel dentro do sistema.

3.2 Classes Cliente, DonoEmpresa e Entregador

As classes Cliente, DonoEmpresa e Entregador herdam de Usuario.

A classe Cliente representa o usuário que realiza pedidos. A classe DonoEmpresa representa o usuário responsável pelo cadastro e administração de empresas, possuindo o atributo específico cpf. A classe Entregador, introduzida no Milestone 02, representa o usuário responsável pela realização de entregas, contendo atributos próprios como veículo, placa, lista de empresas vinculadas e estado de disponibilidade para entrega.

A principal vantagem dessa estrutura é o reaproveitamento de atributos e comportamentos comuns, evitando duplicação de código e fortalecendo o uso de herança no projeto.

3.3 Classe Empresa

A classe Empresa representa os estabelecimentos cadastrados no sistema. No Milestone 02, ela foi transformada em classe abstrata, passando a representar o conceito geral de empresa e servindo como base para os diferentes tipos concretos existentes no sistema.

Ela armazena os atributos comuns a qualquer empresa:

- id
- nome
- endereco
- idDono
- lista de identificadores de produtos
- lista de identificadores de entregadores vinculados

Essa classe participa de relações de composição tanto com produtos quanto com entregadores, pois cada empresa pode possuir vários produtos cadastrados e vários entregadores associados.

3.4 Classes Restaurante, Mercado e Farmacia

A partir do Milestone 02, o sistema passou a trabalhar com subclasses específicas de empresa:

- Restaurante
- Mercado
- Farmacia

A classe Restaurante mantém o atributo tipoCozinha. A classe Mercado adiciona os atributos abre, fecha e tipoMercado. A classe Farmacia adiciona os atributos aberto24Horas e numeroFuncionarios.

Essa modelagem permite especializar corretamente os diferentes estabelecimentos do sistema, aplicando abstração e herança de forma mais adequada ao domínio.

3.5 Classe Produto

A classe Produto representa os itens que podem ser cadastrados em uma empresa e posteriormente adicionados a pedidos. Seus principais atributos são:

- id
- nome
- valor
- categoria
- idEmpresa

Além dos getters, a classe possui setters para permitir atualização de dados do produto, como nome, valor e categoria.

3.6 Classe Pedido

A classe Pedido representa os pedidos realizados pelos clientes. Ela armazena:

- numero
- idCliente
- idEmpresa
- estado
- lista de ids de produtos

Inicialmente, o pedido é criado com o estado "aberto", podendo posteriormente assumir estados como "preparando", "pronto", "entregando" e "entregue", conforme a evolução do fluxo no sistema.

Essa expansão de estados foi importante no Milestone 02, pois permitiu integrar os pedidos ao processo de entrega.

3.7 Classe Entrega

A classe Entrega, adicionada no Milestone 02, representa o processo de entrega de um pedido. Ela armazena informações como:

- id
- nome do cliente
- nome da empresa
- número do pedido
- id do entregador
- nome do entregador
- destino
- lista de produtos

Essa classe foi criada para representar de forma explícita o vínculo entre pedido e entregador, permitindo que o sistema acompanhe e consulte os dados da entrega de forma organizada.

3.8 Classe DadosSistema

A classe `DadosSistema` funciona como um agregador central dos dados do sistema. Nela estão armazenados:

- mapa de usuários;
- mapa de empresas;
- mapa de produtos;
- mapa de pedidos;
- mapa de entregas;
- contadores dos próximos ids e números gerados.

Essa classe é importante porque concentra o estado geral da aplicação e facilita a persistência em arquivo, já que todo o sistema pode ser salvo e recuperado a partir desse único objeto.

3.9 Classe `PersistenciaService`

A classe `PersistenciaService` é responsável por salvar, carregar e apagar os dados persistidos do sistema. A persistência foi feita em arquivo local, garantindo que o sistema mantenha seu estado entre execuções e atenda aos testes de persistência exigidos no projeto.

Essa classe isola a lógica de entrada e saída de arquivo, evitando que esse comportamento fique espalhado pelo restante do código.

3.10 Classes de serviço

O sistema foi dividido em cinco serviços principais:

- `UsuarioService`
- `EmpresaService`
- `ProdutoService`
- `PedidoService`
- `EntregaService`

Cada um desses serviços contém as regras de negócio referentes à sua área.

`UsuarioService`

Responsável por:

- criação de clientes;
- criação de donos de empresa;
- criação de entregadores;
- validação de dados;
- login;
- consulta de atributos de usuário.

`EmpresaService`

Responsável por:

- criação de restaurantes;
- criação de mercados;
- criação de farmácias;
- listagem de empresas por dono;
- consulta de atributos de empresa;
- busca de empresa por nome e índice;
- alteração de funcionamento de mercados;
- vínculo de entregadores com empresas;
- consulta de entregadores e empresas vinculadas.

ProdutoService

Responsável por:

- criação de produtos;
- edição de produtos;
- consulta de atributos de produto;
- listagem de produtos por empresa.

PedidoService

Responsável por:

- criação de pedidos;
- adição de produtos em pedidos;
- consulta de atributos de pedidos;
- fechamento de pedidos;
- liberação de pedidos;
- remoção de produtos;
- busca de pedidos por cliente e empresa.

EntregaService

Responsável por:

- obtenção de pedidos disponíveis para entrega;
- criação de entregas;
- consulta de atributos de entrega;
- busca de entrega por pedido;
- finalização de entregas.

3.11 Classe MyFoodException

A classe `MyFoodException` é uma exceção personalizada criada para representar erros de regra de negócio do sistema, como:

- email duplicado;
- nome inválido;
- produto inexistente;
- empresa não cadastrada;

- tentativa de criar pedido inválido;
- tentativa de criar entrega inválida;
- placa inválida;
- entre outros.

Seu uso melhora a clareza do código, pois permite representar erros do próprio domínio da aplicação de maneira mais organizada.

3.12 Classe Facade

A classe Facade é o ponto de entrada principal do sistema. Ela é a classe utilizada pelos testes de aceitação e funciona como uma interface simplificada para o restante da aplicação.

Seu papel não é concentrar toda a lógica, mas sim:

- inicializar o sistema;
- delegar chamadas para os serviços corretos;
- oferecer os métodos públicos esperados pelos testes;
- controlar operações gerais, como zerar e encerrar o sistema.

3.13 Classe Main

A classe Main é responsável por executar os arquivos de teste utilizando o EasyAccept. Ela não contém regra de negócio do sistema, servindo apenas como ponto de execução para validação das funcionalidades implementadas.

4. Interações entre os componentes

O funcionamento do sistema segue um fluxo bem definido.

Os testes de aceitação executam métodos da classe Facade. A Facade, por sua vez, encaminha cada operação para o serviço correspondente. Os serviços manipulam os objetos armazenados em DadosSistema. Quando necessário, o sistema usa PersistenciaService para salvar ou carregar os dados em arquivo.

Com a introdução do Milestone 02, esse fluxo foi ampliado para incorporar novas interações. Por exemplo, o UsuarioService cria entregadores, o EmpresaService os vincula às empresas, o PedidoService gerencia a transição dos estados do pedido e o EntregaService assume o controle das operações relacionadas à entrega.

De forma resumida, o fluxo principal pode ser descrito assim:

Testes → Facade → Services → DadosSistema / Entidades → Persistência

Essa organização reduz o acoplamento entre os testes e as regras internas do sistema, além de facilitar a manutenção e o entendimento do projeto.

5. Conceitos de Programação Orientada a Objetos aplicados

5.1 Encapsulamento

Os atributos das classes foram definidos como `private`, sendo acessados por métodos públicos apropriados. Isso impede acesso indevido ao estado interno dos objetos e contribui para manter a consistência dos dados.

5.2 Herança

Foi aplicada herança entre `Usuario`, `Cliente`, `DonoEmpresa` e `Entregador`, bem como entre `Empresa`, `Restaurante`, `Mercado` e `Farmacia`. Isso permitiu reutilizar estruturas comuns e especializar apenas o que fosse específico de cada tipo.

5.3 Classe abstrata

As classes `Usuario` e `Empresa` foram implementadas como abstratas, pois representam ideias gerais do sistema, mas não tipos concretos que façam sentido serem instanciados diretamente. Essa decisão reforça o conceito de abstração estudado em sala.

5.4 Polimorfismo

Métodos abstratos como `ehDono()` e `ehEntregador()` foram sobrescritos nas subclasses de usuário, permitindo comportamentos diferentes conforme o tipo concreto do objeto. De forma semelhante, o método `getTipoEmpresa()` diferencia os tipos de empresa.

5.5 Composição

As classes `Empresa`, `Pedido`, `Entregador` e `Entrega` mantêm listas ou referências relacionadas a outros elementos do sistema. Isso representa relações do tipo “tem-vários”, trabalhando composição entre objetos.

5.6 Uso de listas

Foram utilizadas listas para armazenar coleções de ids e relacionamentos, especialmente em empresas, pedidos, entregadores e entregas. Isso está alinhado com o conteúdo visto sobre listas em Java.

5.7 Tratamento de exceções

O sistema utiliza exceções para sinalizar erros de validação e de regras de negócio. Isso evita retornos inconsistentes e torna a lógica de erro mais organizada e explícita.

6. Padrão de projeto adotado

6.1 Nome do padrão: Facade

6.2 Descrição geral

O padrão Facade fornece uma interface simplificada para um conjunto mais complexo de classes e subsistemas. Em vez de o código externo interagir diretamente com diversas classes internas, ele passa a interagir com uma única classe centralizadora.

6.3 Problema resolvido

No contexto do projeto, os testes de aceitação precisam acessar o sistema por meio de uma única classe, sem conhecer sua estrutura interna. Se os testes precisassem interagir diretamente com usuários, empresas, produtos, pedidos, entregadores, entregas e serviços, o sistema se tornaria mais acoplado e difícil de usar externamente.

6.4 Identificação da oportunidade

A oportunidade de aplicar esse padrão surgiu pela própria exigência do projeto, que demanda uma classe Facade compatível com o EasyAccept. Assim, havia a necessidade de uma interface única e simplificada para expor as operações do sistema.

6.5 Aplicação no projeto

A classe Facade foi implementada como ponto de acesso central. Ela expõe os métodos públicos esperados pelos testes, como criação de usuários, login, criação de empresa, criação de produto, criação de pedido, cadastro de entregadores, vínculo com empresas, criação de entrega e assim por diante. Internamente, ela delega a responsabilidade para os serviços apropriados, mantendo a interface externa simples e o design interno mais organizado.

7. Persistência de dados

A persistência do sistema foi implementada em **arquivo local**, garantindo que os dados permaneçam salvos entre diferentes execuções do programa. Essa funcionalidade foi necessária para atender às histórias de usuário relacionadas à persistência nos dois milestones implementados.

A classe PersistenciaService foi responsável por:

- carregar os dados previamente salvos;
- salvar o estado atual do sistema;
- apagar os dados quando o sistema é reiniciado.

Essa abordagem foi suficiente para o escopo do projeto e compatível com os testes de aceitação fornecidos.

8. Testes executados

Foram executados com sucesso os seguintes testes de aceitação dos Milestones 01 e 02:

- us1_1
- us1_2
- us2_1
- us2_2
- us3_1
- us3_2
- us4_1
- us4_2
- us5_1
- us5_2
- us6_1
- us6_2
- us7_1
- us7_2
- us8_1
- us8_2

Todos os testes foram concluídos com sucesso, comprovando que o sistema implementado atende aos requisitos esperados para essas duas etapas do projeto.

9. Conclusão

O projeto MyFood – Milestones 01 e 02 foi desenvolvido com sucesso em Java, atendendo ao escopo previsto para a disciplina de Programação 2. A solução implementada foi compatível com os testes de aceitação até a US 8_2 e permitiu aplicar, na prática, conceitos importantes de programação orientada a objetos, como encapsulamento, herança, abstração, polimorfismo, composição, uso de listas, exceções e organização modular do código.

Além do atendimento funcional, o sistema foi estruturado de forma clara, com separação de responsabilidades entre entidades, serviços, persistência e fachada principal, o que contribuiu para a legibilidade e manutenção do projeto. Com a ampliação realizada no Milestone 02, o sistema passou a incluir novos tipos de empresa, entregadores e entregas, tornando a solução mais completa e mais próxima de um sistema real de gerenciamento de pedidos e entregas.